

PROBLEMAS AL UTILIZAR EL PROGRAMA

1. Registrar Entrada/Salida (Crear)

1.1. No se comprueba si el ID del usuario ya existe para la fecha actual.

```
{
  "id": 1,
  "empleado": "DDD123",
  "fecha": "2026-09-10",
  "entrada": "12:12",
  "salida": "12:12",
  "horas_trabajadas": 0.0
},
{
  "id": 2,
  "empleado": "DDD123",
  "fecha": "2026-09-10",
  "entrada": "23:23",
  "salida": "08:02",
  "horas_trabajadas": -15.35
}
```

Propuesta de mejora:

Existen tres enfoques posibles:

1. Enfoque sencillo:

Se considera que se trata de un trabajo de oficina y que toda la jornada laboral se desarrolla dentro del mismo día. Por lo tanto, cada usuario puede tener un solo registro por día.

2. Enfoque intermedio:

Se permite que un usuario tenga varios registros durante el mismo día. En este caso, se debe comprobar que salida > entrada y que los intervalos de entrada/salida de un registro no se solapen con los intervalos de otros registros del mismo día.

3. Nivel Final Boss:

Se permite que un usuario tenga varios registros, incluidos aquellos que implican un cambio de fecha. En este caso, es necesario: 1) comprobar los solapamientos al crear un nuevo registro; 2) preguntar si se trata de un turno nocturno cuando salida < entrada; 3) gestionar correctamente el cambio de fecha en los turnos nocturnos; 4) al modificar un registro existente, comprobar que el nuevo intervalo no se solape con ninguno de los demás registros del usuario, si existen.

- **Para nivel 1:** Si ya existe un registro para el usuario y la fecha indicados, se debe mostrar dicho registro al usuario y ofrecerle la posibilidad de modificarlo. Si el usuario acepta, se actualizan los datos del registro existente utilizando los datos introducidos. Si no acepta, se mantiene el registro existente y se informa al usuario de que los datos introducidos **NO se han guardado**.
- **Para niveles 2 y 3:** El usuario puede tener más de un registro en el mismo día, pero los intervalos horarios no deben solaparse. Es necesario realizar una comprobación para evitar el solapamiento de horarios.



```
{
  "id": 1,
  "empleado": "EMP001",
  "fecha": "2026-09-10",
  "entrada": "08:00",
  "salida": "10:00",
  "horas_trabajadas": 2.0
}
```



```
{
  "id": 1,
  "empleado": "EMP001",
  "fecha": "2026-09-10",
  "entrada": "08:00",
  "salida": "10:00",
  "horas_trabajadas": 2.0
}
```

```

}
{
  "id": 2,
  "empleado": "EMP001",
  "fecha": "2026-09-10",
  "entrada": "15:00",
  "salida": "23:00",
  "horas_trabajadas": 8.0
}

```

```

}
{
  "id": 2,
  "empleado": "EMP001",
  "fecha": "2026-09-10",
  "entrada": "09:00",
  "salida": "12:00",
  "horas_trabajadas": 3.0
}

```

- **Para niveles 2 y 3:** Si, a partir de los datos introducidos, el sistema detecta un solapamiento de horarios, muestra el registro que genera el conflicto y solicita al usuario que realice una corrección, ya sea modificando el registro existente o introduciendo nuevos datos correctos (aquí simplemente se informa de que es necesario realizar una corrección, sin entrar en la lógica interna de los ciclos).

1.2. Si algún formato de los datos es incorrecto (por ejemplo, las horas), el programa vuelve al inicio, a la selección de opciones.

Propuesta de mejora:

- Si los datos introducidos no cumplen con el formato establecido, el sistema debe ofrecer al usuario la posibilidad de corregirlos en lugar de volver al menú. En el código es necesario implementar las comprobaciones correspondientes y establecer la lógica de repetición necesaria.

1.3. Si la hora de salida es anterior a la hora de entrada, el programa muestra un número negativo de horas.

```

{
  "id": 2,
  "empleado": "DDD123",
  "fecha": "2026-09-10",
  "entrada": "23:23",
  "salida": "08:02",
  "horas_trabajadas": -15.35
}

```

Propuesta de mejora:

- **Para nivel 1:** Basta con comprobar que `salida > entrada`. Si no se cumple esta condición, se debe informar al usuario del error y ofrecerle volver a introducir las horas de `entrada/salida`.
- **Para nivel 2:** Comprobar que `salida > entrada`. Si no se cumple esta condición, se debe informar al usuario del error y ofrecerle volver a introducir las horas de `entrada/salida`.
- **Para nivel 2:** Comprobar que no existan solapamientos con otros registros del mismo usuario y de la misma fecha.
- **Para nivel 3:** Al comprobar los datos introducidos, si la hora de salida es anterior a la hora de entrada, el sistema debe preguntar si se trata de un turno nocturno (con cambio de fecha). Si el usuario confirma que es un turno nocturno, se aceptan los datos entendiendo que la salida corresponde al día siguiente. En caso contrario, se informa al usuario de que los datos introducidos son incorrectos, ya que la hora de salida no puede ser anterior a la hora de entrada si no se trata de un turno nocturno.
- **Para nivel 3:** Si se contempla la posibilidad de realizar turnos nocturnos (con cambio de fecha), es necesario: 1) Preguntar al usuario si se trata de un turno nocturno. 2) Si el usuario confirma que es un turno nocturno, aplicar una fórmula de cálculo específica para las horas trabajadas, teniendo en cuenta el cambio de fecha y evitando resultados negativos. 3) **Importante:** Si el usuario indica que se trata de un turno nocturno, **NO PUEDE introducir la**

hora de salida, ya que la salida tendrá lugar al día siguiente y el usuario todavía no conoce la hora exacta. En este caso, el campo "salida" debe establecerse automáticamente como "Pendiente".

2. Ver Registros (Leer)

2.1. Si hay más de 9 registros, el formato de la tabla se desajusta. También es necesario garantizar un ancho uniforme para los registros de las demás columnas, para conseguir una presentación más ordenada y visualmente suficiente.

ID: 4	Empleado: DDD222	Fecha: 2026-09-10	Entrada: 12:12	Salida: 12:43	Horas: 0.52
ID: 5	Empleado: DDD111	Fecha: 2026-09-10	Entrada: 12:12	Salida: Pendiente	Horas: 0.0
ID: 6	Empleado: DDD222	Fecha: 2026-09-10	Entrada: 12:12	Salida: Pendiente	Horas: 0.0
ID: 7	Empleado: SSS222	Fecha: 2026-09-10	Entrada: 12:22	Salida: Pendiente	Horas: 0.0
ID: 8	Empleado: DDD222	Fecha: 2026-09-10	Entrada: 12:00	Salida: Pendiente	Horas: 0.0
ID: 9	Empleado: WQW222	Fecha: 2026-09-10	Entrada: 12:11	Salida: Pendiente	Horas: 0.0
ID: 10	Empleado: XDD111	Fecha: 2026-09-10	Entrada: 21:23	Salida: Pendiente	Horas: 0.0

Propuesta de mejora:

- Corregir el formato en la función **leer_registros()** para que el ancho de cada columna se ajuste a la longitud del registro más largo de esa columna.

3. Actualizar Registro

3.1. No funciona

Propuesta de mejora:

- Corregir la función **actualizar_registro()**:
 - 1) Eliminar la conversión de tipo incorrecta en la línea **reg_id = int(input("Ingrese el ID del registro a actualizar: "))**;
 - 2) Utilizar la validación del ID mediante la función **validar_id_empleado** y gestionar los resultados no válidos, ofreciendo al usuario volver a introducir el ID;
 - 3) Corregir la validación de la hora: si la hora introducida no es válida, ofrecer al usuario la posibilidad de introducirla de nuevo. También se puede añadir la opción de mantener la hora anterior pulsando ENTER; en este caso, no será necesaria una validación adicional de la hora;
 - 4) (Para los **niveles 2 y 3**) Es necesario comprobar que no existan solapamientos con otros registros. Si se detecta un solapamiento, se debe informar al usuario y volver al formulario para introducir los datos de nuevo.

4. Eliminar Registro

4.1. No funciona

Propuesta de mejora:

- Corregir la función **eliminar_registro()**:
 - 1) Eliminar la conversión de tipo incorrecta en la línea **reg_id = int(input("Ingrese el ID del registro a eliminar: "))**;
 - 2) Utilizar la validación del ID mediante la función **validar_id_empleado** y gestionar los resultados no válidos, ofreciendo al usuario volver a introducir el ID;
 - 3) (Para los **niveles 2 y 3**) Es necesario mostrar la lista de registros correspondientes al ID indicado y ofrecer al usuario la posibilidad de eliminar todos los registros o únicamente uno en concreto (utilizando el campo **"id" del archivo JSON**; no debe confundirse con el **ID del empleado**, que en el archivo JSON se identifica mediante el campo **"empleado"**). También es obligatorio gestionar correctamente las opciones introducidas de forma incorrecta.
 - 4) Después de seleccionar un registro concreto para eliminar, se debe mostrar nuevamente dicho registro al usuario y solicitar su confirmación antes de proceder con la eliminación. Si el

usuario confirma la eliminación, se actualizan los datos del archivo JSON. En caso contrario, se vuelve al menú principal.

5. Salir

5.1. Al seleccionar una opción, el programa no solicita confirmación al usuario y simplemente finaliza la ejecución.

Propuesta de mejora:

- Modificar en la función **menu()** la opción de salida del programa: solicitar al usuario confirmación antes de salir. Si confirma, finalizar el programa y limpiar la pantalla; en caso contrario, volver al menú principal.

OTRAS MEJORAS DEL CÓDIGO

1. Documentar bien las funciones de la siguiente manera:

```
"""_summary_
```

Args:

```
    entrada (_type_): _description_
```

```
    salida (_type_): _description_
```

Returns:

```
    _type_: _description_
```

```
"""
```

2. Separar el comportamiento de la función **cargar_datos()** según el tipo de error. En particular, si se produce un **FileNotFoundError**, la función debe devolver una lista vacía. Sin embargo, si se produce un **json.JSONDecodeError**, se debe informar al usuario de que el archivo está dañado y que el programa no puede ejecutarse correctamente. A continuación, se debe finalizar la ejecución del programa, ya que no tiene sentido continuar trabajando con datos corruptos.
3. En la función **guardar_datos(datos)**, añadir un bloque **try/except** para gestionar posibles errores durante el proceso de guardado de los datos.
4. **OPTIONAL** En la función **validar_formato_hora(hora)**, sería mejor modificar el patrón **patron = r"^([01]\d|2[0-3]):([0-5]\d)\$"** a **patron = r"([01]\d|2[0-3]):([0-5]\d)"** y sustituir **re.match(patron, hora)** por **re.fullmatch(patron, hora)**.
5. **OPTIONAL** En la función **validar_id_empleado(emp_id)**, sería mejor modificar el patrón **patron = r"^([A-Z]{3}\d{3})\$"** a **patron = r"[A-Z]{3}\d{3}"** y sustituir **re.match(patron, emp_id)** por **re.fullmatch(patron, emp_id)**.
6. La lógica de cálculo de las horas trabajadas en la función **calcular_horas(entrada, salida)** debe ajustarse teniendo en cuenta el nivel de desarrollo seleccionado.
7. Todas las funciones que soliciten datos al usuario deben utilizar obligatoriamente **.strip()** para garantizar la eliminación de espacios en blanco innecesarios.
8. Todas las funciones que manipulen con archivos deben utilizar obligatoriamente **try/except** para gestionar posibles errores durante el proceso.

9. El temporizador debería complementarse con un contador regresivo para que el usuario pueda entender qué está sucediendo y saber que el programa no se ha bloqueado. También sería recomendable encapsularlo en una función independiente, por ejemplo:

```
def countdown(seconds: int) -> None:
```

```
    """
```

```
    Muestra un mensaje con la cuenta atrás (en segundos).
```

```
    Actualiza el mensaje cada segundo.
```

```
    Limpia la pantalla una vez finalizada la cuenta atrás.
```

```
    Args:
```

```
    seconds (int): tiempo de espera en segundos.
```

```
    """
```

```
    for i in range(seconds, 0, -1):
```

```
        print(f"\rVolver al menú en {i} seg", end="", flush=True)
```

```
        time.sleep(1)
```

```
    Borro()
```

10. **OPTIONAL** Optimizar los múltiples print del menú en la función **menu()** y sustituirlos por un único bloque de salida:

```
menu = [
```

```
    "1. Registrar Entrada/Salida (Crear)",
```

```
    "2. Ver Registros (Leer)",
```

```
    "3. Actualizar Registro",
```

```
    "4. Eliminar Registro",
```

```
    "5. Salir",
```

```
]
```

```
print("\n--- 🔑 CONTROL DE ASISTENCIA 🔑 ---\n")
```

```
print("\n".join(menu))
```

11. **OPTIONAL** Sustituir los numerosos bloques if/else en la función **menu()** por una estructura match/case:

```
match opcion:
```

```
    case "1":
```

```
        crear_registro()
```

```
    case "2":
```

```
        leer_registros()
```

```
    case "3":
```

```
        actualizar_registro()
```

```
    case "4":
```

```
        eliminar_registro()
```

```
    case "5":
```

```
        Borro() # Borramos pantalla antes de salir.
```

```
        break
```

```
    case _:
```

```
        print("Opción inválida en el menú.")
```

12. En la función **crear_registro()**, sustituir la asignación poco fiable **nuevo_id = 1 if not datos else datos[-1]["id"] + 1** por **nuevo_id = max((r["id"] for r in datos), default=0) + 1**, para garantizar la correcta generación del ID independientemente del orden de los registros.
13. Realizar una validación mínima de la estructura del archivo JSON en la función `cargar_datos()`. Para ello, utilizar la siguiente lógica:

```
def cargar_datos():
    try:
        with open(FILE_NAME, "r", encoding="utf-8") as f:
            datos = json.load(f)

        if not isinstance(datos, list):
            print("❌ La estructura del archivo JSON no es válida.")
            return None

        return datos

    except FileNotFoundError:
        return []

    except json.JSONDecodeError:
        print("❌ El archivo JSON está dañado.")
        return None
```

En las funciones que utilizan **datos = cargar_datos()** (`crear_registro()`, `leer_registros()`, `actualizar_registro()` y `eliminar_registro()`), añadir la comprobación:

```
datos = cargar_datos()
if datos is None:
    return
```

De esta forma, si el archivo JSON tiene una estructura no válida o está dañado, las funciones que dependen de `datos` no continuarán ejecutándose con información incorrecta.

14.